

Putting it together: messy data → summary table

Wednesday, June 3, 2026

i Today: Wednesday June 3

Before class

- **No new reading** — bring the six verbs from Monday and Tuesday

Plan for today

- No new verbs. Today is about **combining and sequencing** them
- The workflow: question → target table → plan → build → **check**
- Working on a genuinely messy dataset (**flights**)
- Not getting fooled: missing values, tiny groups, wrong order

Helpful links

- [Syllabus](#) · [AI policy](#) · [Schedule](#) · [R4DS Ch. 3](#) · [DC Ch. 7](#) · [DC Ch. 10](#) · [dplyr reference](#)

What you already have

Two days in, you have the **whole single-table toolkit**:

Table 1: Six verbs, plus `count()` and the pipe.

Touches...	Verbs
Rows	<code>filter()</code> , <code>arrange()</code>
Columns	<code>select()</code> , <code>mutate()</code>
Groups	<code>group_by()</code> , <code>summarise()</code>

💡 Tip

Today there is **nothing new to memorize**. The skill we're building is *choosing which verb, in what order, and checking that the answer is real*.

The workflow

This is Monday's habit (DC §7.1), now run end to end:

1. **State the question** in plain English.
2. **Sketch the target table:** what does one row represent? what columns?
3. **Plan the verbs** as English sentences, one per step.
4. **Build the pipeline** one verb at a time, looking at the output each time.
5. **Check it.** Does the count make sense? Are missing values handled? Does the number pass a smell test?

i Note

Step 5 is the one people skip, and it's the one Friday's AI lesson was about. A pipeline that *runs* is not the same as a pipeline that's *right*.

A messier dataset

Meet flights

You saw `flights` on HW2 and Friday; it's also the running example in R4DS Ch. 3. Every flight out of NYC in 2013:

```
glimpse(flights)
```

```
Rows: 336,776
```

```
Columns: 19
```

```
$ year      <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2~
$ month     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ day       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ dep_time  <int> 517, 533, 542, 544, 554, 554, 555, 557, 557, 558, 558, ~
$ sched_dep_time <int> 515, 529, 540, 545, 600, 558, 600, 600, 600, 600, 600, ~
$ dep_delay <dbl> 2, 4, 2, -1, -6, -4, -5, -3, -3, -2, -2, -2, -2, -2, -
1~
```

```

$ arr_time      <int> 830, 850, 923, 1004, 812, 740, 913, 709, 838, 753, 849, ~
$ sched_arr_time <int> 819, 830, 850, 1022, 837, 728, 854, 723, 846, 745, 851, ~
$ arr_delay     <dbl> 11, 20, 33, -18, -25, 12, 19, -14, -8, 8, -2, -3, 7, -
1~
$ carrier       <chr> "UA", "UA", "AA", "B6", "DL", "UA", "B6", "EV", "B6", "~
$ flight        <int> 1545, 1714, 1141, 725, 461, 1696, 507, 5708, 79, 301, 4~
$ tailnum       <chr> "N14228", "N24211", "N619AA", "N804JB", "N668DN", "N394~
$ origin        <chr> "EWR", "LGA", "JFK", "JFK", "LGA", "EWR", "EWR", "LGA", ~
$ dest         <chr> "IAH", "IAH", "MIA", "BQN", "ATL", "ORD", "FLL", "IAD", ~
$ air_time      <dbl> 227, 227, 160, 183, 116, 150, 158, 53, 140, 138, 149, 1~
$ distance      <dbl> 1400, 1416, 1089, 1576, 762, 719, 1065, 229, 944, 733, ~
$ hour          <dbl> 5, 5, 5, 5, 6, 5, 6, 6, 6, 6, 6, 6, 6, 6, 5, 6, 6, 6~
$ minute        <dbl> 15, 29, 40, 45, 0, 58, 0, 0, 0, 0, 0, 0, 0, 0, 59, 0~
$ time_hour     <dtm> 2013-01-01 05:00:00, 2013-01-01 05:00:00, 2013-01-
01 0~

```

Where the mess hides

This is what real data looks like after `read_csv("something.csv")`:

- **336,776 rows, 19 columns** — `select()` and `n()` earn their keep here in a way they didn't on penguins.
- **Missing values that mean something:** cancelled flights have NA for `dep_delay` and `arr_delay`.
- **Codes, not labels:** carrier is "UA", not "United". (Turning codes into names needs a *join*, which is later, so today we'll live with the codes.)

```

flights |>
  summarise(
    n           = n(),
    missing_dep = sum(is.na(dep_delay))
  )

```

```

# A tibble: 1 x 2
      n missing_dep
  <int>      <int>
1 336776      8255

```

Warning

Over 8,000 rows have a missing departure delay. Every average we compute has to decide what to do with them, or it silently returns NA.

Worked example 1

Q: which months had the worst delays?

English plan:

- One row per **month**, with its average departure delay and a count.
- *Group by month, then summarise the mean and n(), then arrange worst-first.*

Target table: month | avg_delay | n — twelve rows.

```
flights |>
  group_by(month) |>
  summarise(
    avg_delay = mean(dep_delay, na.rm = TRUE),
    n         = n(),
    .groups   = "drop"
  ) |>
  arrange(desc(avg_delay))
```

```
# A tibble: 12 x 3
  month avg_delay    n
  <int>   <dbl> <int>
1     7    21.7 29425
2     6    20.8 28243
3    12    16.6 28135
4     4    13.9 28330
5     3    13.2 28834
6     5    13.0 28796
7     8    12.6 29327
8     2    10.8 24951
9     1    10.0 27004
10    9     6.72 27574
11   10     6.24 28889
12   11     5.44 27268
```

What just happened

We used **three** verbs in sequence, and two safety habits from this week:

- `na.rm = TRUE` so the cancelled flights don't poison the mean (*Monday*)
- `n = n()` so we can see each average rests on ~25,000 flights, not a handful (*Monday*)

Tip

Summer months (6, 7) and December rise to the top — which matches what you'd expect from thunderstorms and holiday travel. **That agreement is the smell test passing.** If January had come out worst by a mile, you'd go looking for a bug.

Worked example 2

Q: worst JFK summer destinations?

Question: for flights leaving **JFK in summer**, which **destination** had the worst average arrival delay, among destinations with **enough flights to trust**?

English plan:

- *Filter* to JFK, summer months.
- *Group by* destination, *summarise* mean arrival delay + `n()`.
- *Filter again* to drop tiny destinations.
- *Arrange* worst-first.

Build it one verb at a time

```
flights |>
  filter(origin == "JFK", month %in% c(6, 7, 8)) |>
  group_by(dest) |>
  summarise(
    avg_arr_delay = mean(arr_delay, na.rm = TRUE),
    n              = n(),
    .groups        = "drop"
  ) |>
  filter(n >= 100) |>
  arrange(desc(avg_arr_delay))
```

```
# A tibble: 49 x 3
  dest   avg_arr_delay     n
  <chr>         <dbl> <int>
1 CMH           44.4   194
2 CVG           39.4   267
3 IND           30.8   144
4 PDX           29.6   265
5 BNA           28.8   184
6 DEN           28.6   184
7 CLE           28.2   199
8 ORF           26.7   128
9 PBI           25.9   368
10 JAX           22.1   365
# i 39 more rows
```

Note

Notice `filter()` appears **twice**, and it's the *same verb* doing two different jobs: once on the raw rows (which flights), once on the summary table (which destinations are big enough to believe). Same tool, different point in the pipeline.

Why the second filter matters

Without `filter(n >= 100)`, the “worst destination” is whichever one had a single very-late flight. That’s exactly Monday’s lesson: **a mean from a group of 3 is not a fact.**

```
# Drop the n >= 100 line and the top of the list is full of
# destinations with n = 1 or n = 2. Impressive-looking, meaningless.
```

This is also a verification move: the count column is how you *catch* the problem before it ends up in a report.

Worked example 3

Q: do long hauls leave later?

Now bring in `mutate()` and end with a plot.

English plan: *mutate* a long/short label from **distance**, *group by* it, *summarise* the average delay, *then plot* the summary table.

```

flights |>
  mutate(haul = ifelse(distance > 1000, "long", "short")) |>
  group_by(haul) |>
  summarise(
    avg_delay = mean(dep_delay, na.rm = TRUE),
    n         = n(),
    .groups   = "drop"
  )

```

```

# A tibble: 2 x 3
  haul avg_delay     n
<chr>   <dbl> <int>
1 long     11.5 147105
2 short    13.5 189671

```

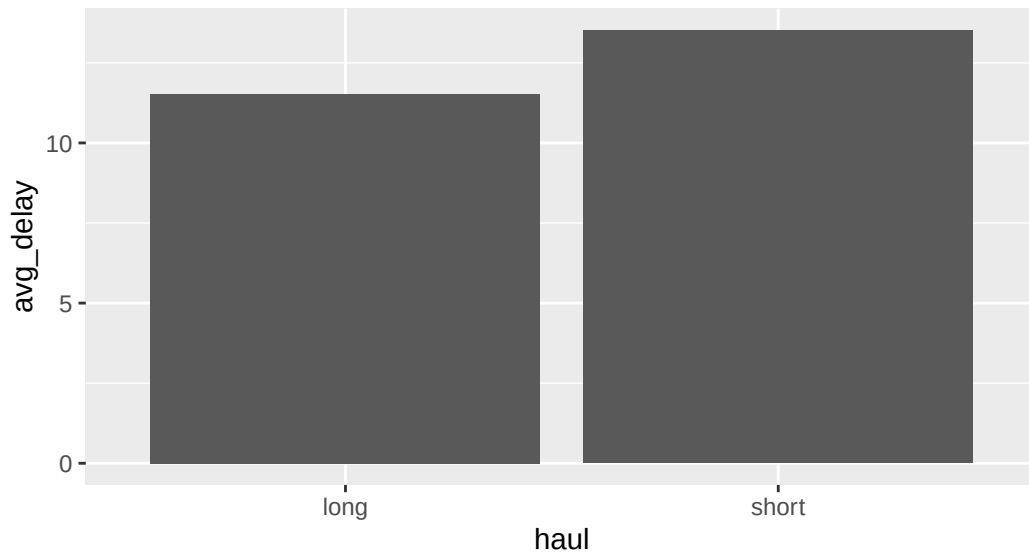
From summary table to plot

Because the summary is just a (tiny) data frame, it's already glyph-ready, pipe it into `ggplot()`, switching `|>` to `+`:

```

flights |>
  mutate(haul = ifelse(distance > 1000, "long", "short")) |>
  group_by(haul) |>
  summarise(avg_delay = mean(dep_delay, na.rm = TRUE), .groups = "drop") |>
  ggplot(aes(x = haul, y = avg_delay)) +
  geom_col()

```



This is the full arc of the week: **raw rows** → **wrangle to a glyph-ready table** → **graph it**.

Getting it right

Order matters

The same verbs in a different order answer different questions:

Filter, *then* summarise

“Average delay *of JFK flights*.”

```
flights |>
  filter(origin == "JFK") |>
  summarise(
    avg = mean(dep_delay,
              na.rm = TRUE)
  )
```

Summarise, *then* filter

“Of the per-origin averages, keep JFK’s.”


```
flights |>
  group_by(origin) |>
  summarise(
    avg = mean(dep_delay,
               na.rm = TRUE)
  ) |>
  filter(origin == "JFK")
```

Both are valid. The trick is knowing **which question you’re actually asking** before you write it.

The three checks

Before you believe any wrangled result, run the checklist that’s been building all week:

Table 2: Your verification checklist.

Check	The verb/habit	First seen
Did missing values get handled?	<code>na.rm, !is.na()</code>	Mon / Tue
Is each number based on enough rows?	<code>n = n()</code>	Monday
Does the answer pass a smell test?	compare to what you know	Friday

Warning

A pipeline that runs without error has cleared the *lowest* bar. These three checks are what separate “it ran” from “it’s right.”

What we did today

- **No new verbs** — just the six from Monday and Tuesday, sequenced
- **The workflow:** question → target table → English plan → build incrementally → check
- **Real mess:** missing delays, code-not-name columns, tiny groups
- **filter() in two roles:** subsetting raw rows *and* trimming a summary table
- **Order matters:** filter-then-summarise summarise-then-filter
- **The full arc:** raw rows → glyph-ready summary → plot

Activity

Activity: a flights investigation

Roughly 25 minutes. Individually or with a neighbor.

- [Activity Canvas Link](#)

Format: you'll answer a series of questions about **flights** by building pipelines, but each one starts with writing the **English plan first**. You'll compare two pipelines that look alike but answer different questions, catch a silently-dropped NA, and finish with one open-ended question where you choose the verbs and **report your sanity checks**.

To turn in: save your `.qmd` and rendered HTML.

Wrap-up & looking ahead

Tomorrow (Thursday): new reading and new content, reshaping data between **wide and narrow** with `pivot_longer()` and `pivot_wider()`.

Upcoming Deadlines

- Thu 6/4, before class: [DC §12.1](#)
- Thu 6/4, 11:59 p.m.: [Project: Team formation](#)

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

[Syllabus](#) · [AI policy](#) · [Schedule](#) · [R4DS Ch. 3](#) · [DC Ch. 10](#) · [dplyr reference](#)